

Module 8 – Conclusion & Best Practices

Project conclusion, implementation summary, and best practices

Conclusion

The Implement Client Script & UI Policy (Incident) project demonstrates how ServiceNow client-side controls can improve data quality and form usability. UI Policies dynamically control field behavior, while Client Scripts provide automated updates and validation.

Implementation Summary

High Impact Control UI Policy makes Assignment group mandatory when Impact is High and reverses the behavior when the condition is false.

The Urgency UI Policy Action makes Urgency read-only for High Impact Incidents.

The onChange Client Script automatically sets Urgency to High when Impact becomes High.

The onSubmit Client Script prevents saving a High Impact Incident when Assigned To is empty.

The onCellEdit Client Script blocks direct State changes from Incident list editing.

Benefits

Improves consistency of Incident data.

Reduces incomplete submissions.

Provides immediate feedback to users.

Automates repetitive field updates.

Controls list-based editing for selected fields.

Improves the reliability of triage and reporting data.

Best Practices

Use UI Policies for simple dynamic field behavior such as mandatory, visible, and read-only controls.

Use onChange scripts for client-side reactions to field changes.

Use onSubmit scripts for final client-side form validation.

Keep Client Scripts small, readable, and condition-specific.

Test both the true and reverse conditions of UI Policies.

Test both list editing and normal form updates where onCellEdit is used.

Remember that client-side validation is a user-interface control; server-side validation may be needed when a rule must be enforced regardless of the client.

Final Result

The completed project provides a practical Incident Management example showing how UI Policies and Client Scripts can work together to enforce conditional field behavior, automate values, validate

submissions, and control list editing.